

Segmentation Backend

`bb_utils.segmentation` provides a common interface for running instance and semantic segmentation models on RGB images, normalising each model's output into a single binary mask format that callers (e.g. `bb-run-segmentation`) can consume without knowing which model produced it.

Table of Contents

- [Output interface](#)
 - [Package layout](#)
 - [Architecture](#)
 - [Available backends](#)
 - [YoloBackend](#)
 - [Mask2FormerBackend](#)
 - [CLI usage](#)
 - [Config format](#)
 - [Output files](#)
 - [Mask utilities](#)
 - [Adding a new backend](#)
 - [Dependencies](#)
-

Output contract

Every backend satisfies the same interface defined in `base.SegmentationBackend`:

```
mask = backend.segment(image, target_classes)
```

Property	Value
Input <code>image</code>	<code>np.ndarray</code> , shape <code>(H, W, 3)</code> , dtype <code>uint8</code> , colour order RGB
Input <code>target_classes</code>	<code>List[int]</code> : model-specific class indices to include
Output <code>mask</code>	<code>np.ndarray</code> , shape <code>(H, W)</code> , dtype <code>uint8</code> , values strictly in <code>{0, 1}</code>
Output semantics	<code>1</code> = pixel belongs to a detected instance of a target class; <code>0</code> = background
No detections	Returns an all-zeros mask (valid result, not an error)
Spatial alignment	<code>mask[i, j]</code> corresponds to <code>image[i, j]</code> , no spatial transformation applied

The backend does **not** apply dilation and does **not** convert grayscale input to RGB; both are the caller's responsibility.

Architecture

The package follows a **registry + factory** pattern so that new backends can be added without modifying any caller code.

```

caller
├─ create_backend(config) ← factory.py
│   └─ reads config["model"]["backend"]
│       └─ looks up _BACKEND_REGISTRY[name]
│           └─ instantiates the registered SegmentationBackend subclass
│               └─ .segment(image, target_classes) → uint8 (H, W) mask

```

New backends are registered at import time:

```

from bb_utils.segmentation.factory import register_backend
from my_package.my_backend import MyBackend

register_backend("my_backend", MyBackend)

```

Available backends

YoloBackend

Module: `bb_utils.segmentation.yolo_backend`

Config key: `"yolo"`

Dependency: `ultralytics` (`pip install ultralytics`)

Wraps Ultralytics YOLOv8-seg (or any later `*-seg` variant). Returns the union of all per-instance masks whose predicted class is in `target_classes`.

Class indices follow the model's training dataset. For the default YOLOv8 weights (COCO): class `0 = person`.

Inference flow:

1. `model.predict(image_rgb, conf=..., iou=..., device=...)`, runs NMS internally.
2. For each detected instance whose class is in `target_classes`, extract the float instance mask at model resolution.
3. Threshold at `0.5` → binary; resize to input `(H, W)` with nearest-neighbour if the model output resolution differs.
4. Accumulate via bitwise OR into the output mask.

Config keys (all under `model:`):

Key	Type	Required	Default	Description
<code>backend</code>	str	yes	—	Must be "yolo"
<code>model_name</code>	str	no	"yolov8n-seg"	Model identifier or path accepted by <code>YOLO()</code>
<code>device</code>	str	no	"cpu"	"cuda", "cuda:0", "cpu"
<code>confidence_threshold</code>	float	yes	—	Minimum detection confidence (0, 1)
<code>iou_threshold</code>	float	yes	—	NMS IoU threshold (0, 1)

Mask2FormerBackend

Module: `bb_utils.segmentation.mask2former_backend`

Config key: "mask2former"

Dependencies: `transformers, torch` (`pip install transformers torch`)

Wraps [HuggingFace Mask2Former](#) for instance or panoptic segmentation. Any `Mask2FormerForUniversalSegmentation` checkpoint from the HuggingFace Hub is supported. Weights are downloaded automatically on first use.

Class indices follow COCO (same convention as YOLOv8-seg): class 0 = `person`.

Inference flow:

1. Preprocess image via `AutoImageProcessor`; run model forward pass.
2. Post-process via `post_process_instance_segmentation` or `post_process_panoptic_segmentation` depending on `segmentation_type`.
3. For each segment whose `label_id` is in `target_classes` and score $\geq$ `confidence_threshold`, merge it into the output mask via bitwise OR.

Config keys (all under `model`):

Key	Type	Required	Default	Description
<code>backend</code>	str	yes	—	Must be "mask2former"
<code>model_name</code>	str	no	"facebook/mask2former-swin-tiny-coco-instance"	HuggingFace repo ID or local path
<code>device</code>	str	no	"cpu"	"cuda", "cuda:0", "cpu"
<code>confidence_threshold</code>	float	no	0.5	Minimum prediction score (0, 1)

Key	Type	Required	Default	Description
segmentation_type	str	no	auto	"instance" or "panoptic" — auto-detected from model_name

iou_threshold is not used (Mask2Former does not apply NMS).

Recommended checkpoints for pedestrian masking:

facebook/mask2former-swin-tiny-coco-instance	# fastest
facebook/mask2former-swin-small-coco-instance	
facebook/mask2former-swin-base-coco-instance	
facebook/mask2former-swin-large-coco-instance	# most accurate
facebook/mask2former-swin-large-coco-panoptic	# panoptic; see note below

Note: panoptic checkpoint and safetensors workaround

facebook/mask2former-swin-large-coco-panoptic (and other panoptic checkpoints) ship only pytorch_model.bin, not model.safetensors. Since transformers ≥ 5.6 blocks loading .bin files with torch < 2.6 (CVE-2025-32434), using the Hub ID directly will fail unless torch is upgraded.

Workaround — download the safetensors weights from the model's open PR#3 and point model_name at the local directory:

```
from huggingface_hub import snapshot_download
snapshot_download(
    "facebook/mask2former-swin-large-coco-panoptic",
    revision="refs/pr/3",
    local_dir=~/.cache/huggingface/hub/mask2former-swin-large-coco-panoptic-safetensors",
    ignore_patterns=["*.bin", "*.msgpack", "flax_model*"],
)
```

Then in the config:

```
model_name: "~/.cache/huggingface/hub/mask2former-swin-large-coco-panoptic-safetensors"
segmentation_type: "panoptic"
```

This is machine-specific. The permanent fix is to upgrade torch to ≥ 2.6.

CLI usage

Prerequisites: install the backend dependency before running:

```
pip install ultralytics          # required for YoloBackend (default)
pip install transformers torch   # required for Mask2FormerBackend
```

```
# Segment a directory of images
bb-run-segmentation \
  --config configs/preprocessing_segmentation.yaml \
  --images-dir /path/to/images \
  --output-dir /path/to/masks

# Overwrite already-generated masks
bb-run-segmentation \
  --config configs/preprocessing_segmentation.yaml \
  --images-dir /path/to/images \
  --output-dir /path/to/masks \
  --force

# Validate config and paths without running inference
bb-run-segmentation \
  --config configs/preprocessing_segmentation.yaml \
  --images-dir /path/to/images \
  --output-dir /path/to/masks \
  --dry-run

# For a split-based dataset, call once per split
bb-run-segmentation --config configs/preprocessing_segmentation.yaml \
  --images-dir dataset/incrowdvi/images/train \
  --output-dir dataset/incrowdvi/semantic_masks/train
bb-run-segmentation --config configs/preprocessing_segmentation.yaml \
  --images-dir dataset/incrowdvi/images/val \
  --output-dir dataset/incrowdvi/semantic_masks/val
```

CLI flags:

Flag	Required	Description
<code>--config</code>	yes	Path to segmentation YAML config (model settings)
<code>--images-dir</code>	yes	Flat directory of <code>*.png</code> frames to segment
<code>--output-dir</code>	yes	Destination directory for mask NPZ files
<code>--force</code>	no	Overwrite existing mask NPZ files
<code>--dry-run</code>	no	Log config summary and exit without running inference
<code>--verbose</code>	no	Enable DEBUG-level logging

Output files

The runner writes one compressed NPZ per source frame:

```
{output_dir}/{stem}.npz
```

Each file contains a single array:

Key	dtype	Shape	Description
mask	uint8	(H, W)	1 = detected pedestrian pixel; 0 = static background

The mask is in the same pixel space as the source image (no spatial transformation applied). `mask_dilation_px` is applied before writing to compensate for boundary inaccuracy in fisheye frames.

Loading a mask:

```
import numpy as np

data = np.load("dataset/incrowdvi/semantic_masks/train/frame_stem.npz",
allow_pickle=False)
mask = data["mask"] # uint8 (H, W), values in {0, 1}
```

Visual spot-check

Use the bundled example script to overlay a sample of masks on their source frames and save the blended PNGs for manual inspection:

```
# Default: 20 frames spread across zero/sparse/medium/dense density buckets
python evaluation/segmentation_spotcheck.py \
  --masks-dir dataset/incrowdvi/semantic_masks/train \
  --images-dir dataset/incrowdvi/frames/train \
  --output-dir logs/mask_spotcheck

# Custom sample size, random seed, and overlay opacity
python evaluation/segmentation_spotcheck.py \
  --masks-dir dataset/incrowdvi/semantic_masks/train \
  --images-dir dataset/incrowdvi/frames/train \
  --output-dir logs/mask_spotcheck \
  --n-frames 40 \
  --seed 123 \
  --alpha 0.5
```

Flag	Default	Description
<code>--n-frames</code>	20	Total frames to sample, spread evenly across the four buckets
<code>--seed</code>	42	Random seed for reproducible sampling
<code>--alpha</code>	0.45	Opacity of the red mask overlay (0 = invisible, 1 = solid)
<code>--all</code>	off	Process every mask in <code>--masks-dir</code> instead of sampling

Frames are drawn from four density buckets (`zero`, `sparse`, `medium`, `dense`) so both empty scenes and heavily masked frames are always represented. Output filenames encode the bucket and density value, e.g. `dense_d0.45_Kiko_loop_R_1058664352.png`.

Mask utilities

`bb_utils.segmentation.utils` provides post-processing helpers. All functions accept and return `uint8` (H, W) masks with values in `{0, 1}`.

`dilate_mask(mask, radius)`

Morphological dilation by a circular structuring element of `radius` pixels. Used to compensate for boundary inaccuracy in fisheye segmentation.

- Uses `scipy.ndimage.binary_dilation` when available; falls back to a pure-numpy iterative approach.
- `radius=0` is a no-op.

```
from bb_utils.segmentation.utils import dilate_mask

dilated = dilate_mask(mask, radius=5)
```

`union_masks(masks)`

Elementwise OR of a list of masks (same shape required).

```
from bb_utils.segmentation.utils import union_masks

combined = union_masks([mask_a, mask_b, mask_c])
```

`resize_mask(mask, target_shape)`

Resize a binary mask to (H_{new}, W_{new}) using nearest-neighbour interpolation to preserve binary values.

- Uses `PIL.Image.NEAREST` when Pillow is available; falls back to a pure-numpy index-based approach.

```
from bb_utils.segmentation.utils import resize_mask

resized = resize_mask(mask, (480, 640))
```

Adding a new backend

1. Create a subclass of `SegmentationBackend` in a new file, e.g. `bb_utils/segmentation/sam_backend.py`:

```
from bb_utils.segmentation.base import SegmentationBackend

class SamBackend(SegmentationBackend):
    def __init__(self, model_name, device, **kwargs):
        ...

    def segment(self, image, target_classes):
        # image: uint8 (H, W, 3) RGB
        # must return uint8 (H, W) with values in {0, 1}
        ...
```

2. Register it in `factory.py` (or at import time in your submodule):

```
from bb_utils.segmentation.factory import register_backend
from bb_utils.segmentation.sam_backend import SamBackend

register_backend("sam", SamBackend)
```

3. Use it by setting `model.backend: sam` in the pipeline YAML config.

No changes are required in `segmentation_runner.py` (`bb_utils.data_preparation`) or any other caller.

Dependencies

Dependency	Required by	Install
numpy	all modules	<code>pip install numpy</code>
ultralytics	YoloBackend	<code>pip install ultralytics</code>
transformers	Mask2FormerBackend	<code>pip install transformers</code>
torch	Mask2FormerBackend	<code>pip install torch</code>

Dependency	Required by	Install
scipy	dilate_mask (optional)	pip install scipy
Pillow	resize_mask, Mask2FormerBackend (optional)	pip install Pillow

scipy and Pillow are optional: both dilate_mask and resize_mask fall back to pure-numpy implementations when they are not installed, at a small performance cost for large masks or radii.